Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-1113 Sistemas Empotrados

Documento de Diseño (DI)

Proyecto I — Sistema embebido a la medida para un robot aspiradora autónomo con reproducción de audio y control remoto

Integrantes

Randall Bolaños López	2019043784
Aldo Alessandro Bagliano Gómez	2020018456
Yendry Badilla Gutierrez	2016113047
Christopher David Rodriguez Cordero	2022040771

Profesor

Dr.-Ing. Jeferson González Gómez

Octubre de 2026

Índice

1. Introducción **2**

2. DI1 — Necesidades y requerimientos **2**

2.1. El problema 2

2.2. Partes interesadas y sus necesidades 2

2.3. Requerimientos técnicos 3

2.4. Salud y seguridad pública 4

2.5. Costos 4

2.6. Impacto ambiental 5

2.7. Recursos disponibles y restricciones 5

3. DI2 — Valoración de alternativas **6**

3.1. Método 6

3.2. Detección de obstáculos 6

3.3. Alimentación 6

3.4. Salida de audio 7

3.5. Odometría 7

3.6. Otras decisiones 7

4. DI3 — Diseño de la alternativa seleccionada **8**

4.1. Arquitectura de hardware 8

4.2. Arquitectura de software 9

4.3. Navegación 10

4.4. Elementos propios del diseño 10

4.5. Cómo responde el diseño a seguridad, costo e impacto 11

5. DI4 — Validación del diseño final **11**

5.1. Estrategia 11

5.2. Matriz de cumplimiento 11

5.3. Validación de la seguridad 13

5.4. Validación de la eficiencia 14

5.5. Validación del costo y del impacto 14

5.6. Hallazgos de la validación y acciones correctivas 15

5.7. Limitaciones conocidas 15

6. Conclusiones **16**

7. Referencias **16**

1. Introducción

Este documento evidencia el atributo de **Diseño (DI)** en el Proyecto I del curso. Recorre el proceso completo: qué se necesitaba y bajo qué restricciones (DI1), qué alternativas se valoraron y con qué criterios (DI2), cómo quedó diseñada la alternativa elegida (DI3) y cómo se comprobó que el resultado cumple los requerimientos y las consideraciones de seguridad, costo e impacto (DI4).

El producto es un robot aspiradora de tracción diferencial sobre una Raspberry Pi 4 con una imagen de Linux mínima construida con Yocto (Poky, rama `scarthgap`). Navega de forma autónoma evadiendo obstáculos, reproduce MP3 y se opera desde un panel web que muestra la telemetría y el mapa de recorrido en tiempo real. Todo acceso al hardware pasa por una biblioteca dinámica propia, `librobot.so`, compilada de forma cruzada.

El detalle técnico —cálculos, listas de materiales, procedimientos de verificación y evidencias— está en la carpeta `docs/` del repositorio. Aquí se resume y se cita el archivo correspondiente.

2. DI1 — Necesidades y requerimientos

2.1. El problema

Un robot de limpieza coordina en tiempo real subsistemas que compiten por los mismos recursos: sensado de proximidad, tracción, audio, red e interfaz de usuario. El problema es complejo por tres razones que no se pueden resolver por separado:

- **Recursos acotados.** El enunciado fija un presupuesto de 200 MB de sistema de archivos raíz y 15 s de arranque. Los GPIO entregan 16 mA por pin y 50 mA en total, y no toleran más de 3.3 V.
- **Dominios eléctricos incompatibles.** La lógica trabaja a 3.3 V y miliamperios; los motores, a 9 V con picos de varias veces su corriente nominal. Deben convivir sin compartir tierra.
- **Concurrencia.** Navegar, medir, reproducir audio y atender la interfaz ocurren a la vez, y ninguna tarea puede degradar la respuesta ante un obstáculo.

2.2. Partes interesadas y sus necesidades

Parte interesada	**Necesidad**
Usuario final	Un robot que limpie sin supervisión, que se pueda tomar a mano desde el celular y cuyo estado se entienda de un vistazo (LEDs, sonidos, mapa).
Curso y profesor	Cumplimiento verificable del enunciado, y que el equipo prestado (Raspberry Pi 4 y su kit) regrese sin daño.
Grupo de trabajo	Un diseño que cuatro personas puedan construir en seis semanas, con el inventario disponible y un costo que el grupo pueda asumir.
Terceros en el entorno	Que el robot no represente un riesgo: ni eléctrico, ni de incendio, ni por golpear a personas u objetos, ni por exponer la red.

2.3. Requerimientos técnicos

Los requerimientos obligatorios del enunciado se numeran para trazarlos después en la validación (sección 5.2).

Id.	Área	Requerimiento
R1	Navegación	Modo autónomo con al menos un algoritmo de cobertura reactiva. Ante un obstáculo: detenerse, retroceder y cambiar de dirección.
R2	Sensado	Al menos dos sensores de proximidad para detección frontal y lateral, leídos y procesados en tiempo real.
R3	Tracción	Dos motores DC con control diferencial (avance, retroceso, giros, detención) y velocidad por PWM para giros de radio variable.
R4	Indicadores	Cuatro LEDs: modo autónomo, modo manual, obstáculo y sistema encendido.
R5	Audio	Reproducir MP3 locales de forma concurrente con la navegación.
R6	Audio	Desde la interfaz: elegir canción, reproducir, pausar, detener y ajustar el volumen.
R7	Audio	Salida directa desde el robot, con parlante a bordo.
R8	Audio	Sonidos de notificación en cuatro eventos: inicio del sistema, inicio del modo autónomo, obstáculo y cambio a manual.
R9	Control remoto	Conmutar entre modo autónomo y manual; en manual, los controles direccionales mandan los motores.
R10	Control remoto	Panel con modo activo, controles, sensores en tiempo real, control de audio, estado de los LEDs y mapa.
R11	Mapa	Grilla 2D incremental (visitada, obstáculo, desconocida) a partir de los sensores y la odometría, visible en tiempo real.
R12	Seguridad	Pantalla de inicio de sesión con al menos un usuario y un protocolo de seguridad mínimo.
R13	Arranque	Inicio automático con una unidad `systemd` propia, `Restart=on-failure` y sin interfaz gráfica local.
R14	Biblioteca	Biblioteca dinámica propia que encapsule motores, sensores, LEDs y audio; el servidor la usa de forma exclusiva.
R15	Desarrollo	Todo el software se compila de forma cruzada en el host.
R16	Sistema operativo	Imagen mínima con Yocto, solo con los paquetes necesarios y cada uno justificado.
R17	Construcción	CMake o Autotools, generando el paquete estándar.
R18	Receta	Al menos una receta `.bb` propia; la imagen se reproduce con `bitbake <imagen>` sin pasos manuales.
R19	Eficiencia	Medir y reportar rootfs, tiempo de arranque, RAM y CPU en operación normal. Referencia: rootfs ≤ 200 MB y arranque ≤ 15 s.

Se implementaron además los tres requerimientos opcionales:

- **RO1, desnivel.** Dos sensores infrarrojos al piso en las esquinas delanteras (GPIO 4 y 8)

detienen el robot ante el borde de una grada; en autónomo retrocede y gira hacia el otro lado, y en manual no deja avanzar.

- **RO2, fin de ciclo.** El usuario fija una meta en minutos de modo autónomo o en metros cuadrados recorridos; al cumplirla el robot se detiene, suena un aviso y el panel lo informa.
- **RO3, playlist persistente.** La lista se edita desde el panel y se guarda en la partición de la música, así sobrevive a los reinicios.

2.4. Salud y seguridad pública

Riesgo	**Requerimiento que se deriva**
Incendio o fuga térmica de una celda de Li-Ion	Toda celda de Li-Ion debe llevar protección de sobrecarga, sobredescarga y cortocircuito; nunca alimentar la Raspberry directo de la celda.
Daño al equipo prestado	Aislamiento óptico entre la lógica y la etapa de potencia, con tierras separadas; ninguna señal de 5 V llega a un GPIO.
Golpes a personas u objetos	Freno automático ante un obstáculo también en modo manual; masa menor a 1.5 kg; los motores quedan frenados en reposo y durante el arranque.
Cables en las llantas	Ruteo fijo del cableado, sin tramos sueltos a menos de 1 cm de una llanta.
Acceso no autorizado por la red	Inicio de sesión obligatorio; el demonio de GPIO solo escucha en la interfaz local.
Nivel sonoro	Volumen ajustable desde la interfaz y potencia máxima acotada por el divisor de entrada del amplificador.

2.5. Costos

La Raspberry Pi 4 y su kit los presta el curso, de modo que el costo para el grupo es el del modelo físico. La compra se planeó con una estimación por bloques:

Bloque	**Estimación (USD)**
Alimentación portátil (baterías, protección y regulación)	25–45
Circuitería y sensores (optoacopladores, servo, pasivos, placas)	9–17
Etapa de audio (PAM8403, parlante, filtro)	5–12
Chasis y acabado	10–25
Total del robot	**49–99**

El requerimiento que se fijó fue mantenerse dentro de ese intervalo y aprovechar primero lo que el grupo ya tenía: un sensor ultrasónico, un MPU-6050 y un driver L298N.

El gasto real del grupo fue el siguiente. Las compras se hicieron en colones y se expresan en dólares a un tipo de cambio aproximado de 500 colones por dólar:

Gasto	Costo (USD)
Dos motores y un ventilador del primer diseño, que se descartaron al rehacerlo	24
Dos motores para el diseño final, los que lleva el robot	16
Impresión 3D de las piezas del robot	50
Aspiradora robot usada, para hacerle ingeniería inversa	20
Optoacopladores, placas y otros componentes	10
Otros gastos relacionados	14
Total	**134**

De ese total, 90 USD quedaron en el robot (motores, impresión 3D, circuitería y otros gastos) y 44 USD fueron costo de aprendizaje: los motores y el ventilador del primer diseño, que se dejaron de usar cuando el diseño se rehízo, y la aspiradora que se desarmó para estudiarla. La comparación con la estimación está en la sección 5.5.

2.6. Impacto ambiental

- **Baterías.** Son el componente de mayor impacto. Se prefieren recargables, deben poder desmontarse, y al final de su vida se entregan a un gestor de residuos electrónicos.
- **Residuos electrónicos.** El montaje debe permitir reutilizar los módulos: conectores desconectables y nada pegado de forma permanente a la Raspberry Pi, que se devuelve al curso.
- **Consumo.** Menos consumo es más autonomía con las mismas baterías: se prefiere amplificación clase D y un sistema operativo sin servicios de más.
- **Compras.** Reutilizar el inventario del grupo antes que comprar componentes nuevos.

2.7. Recursos disponibles y restricciones

Recurso	Disponibilidad
Plataforma	Raspberry Pi 4 y kit, prestados por el curso.
Inventario del grupo	Un HC-SR04, un MPU-6050 (módulo GY-521) y un driver L298N de doble puente.
Personas y tiempo	Cuatro integrantes, del 25 de agosto al 6 de octubre de 2026.
Host de construcción	Laptop de 4 núcleos y 7 GB de RAM, por debajo de lo recomendado para Yocto.
Herramientas	Multímetro, cautín, y el Toolchain-SDK generado por el propio proyecto.

De ahí salen dos restricciones que marcaron el diseño: el hardware llegó tarde respecto del software, así que hacía falta poder desarrollar y probar sin él; y el host de construcción era el cuello de botella de cada iteración de la imagen.

3. DI2 — Valoración de alternativas

3.1. Método

Las decisiones de mayor peso se evaluaron con una matriz ponderada. Cada alternativa recibe una nota de 1 (peor) a 5 (mejor) en cuatro factores:

Factor	Peso	Qué mide
Técnico	40 %	Cumplimiento del requerimiento, desempeño, uso de GPIO y riesgo de integración.
Económico	25 %	Costo de compra y disponibilidad local; cuánto del inventario se aprovecha.
Ambiental	15 %	Cantidad de componentes, consumo y reutilización.
Social	20 %	Seguridad de las personas y del equipo prestado.

El factor técnico pesa más porque una alternativa que no cumple el requerimiento no es una alternativa. El social pesa más que el ambiental porque el enunciado hace al grupo responsable del equipo prestado.

3.2. Detección de obstáculos

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Tres HC-SR04 fijos	3	3	3	4	3.20
Un HC-SR04 sobre un servo (radar)	4	4	4	3	**3.80**
Sensores infrarrojos	2	4	3	2	2.65

Los tres sensores fijos dan una lectura frontal rápida ($\approx 16\,\text{Hz}$), pero solo tres direcciones, ocupan seis GPIO, obligan a secuenciar los disparos para evitar ecos cruzados y exigían comprar dos sensores más. Los infrarrojos económicos tienen poco alcance, dependen del color de la superficie y de la luz ambiente, y no entregan una distancia útil para el mapa.

El radar mide siete direcciones con tres GPIO y el sensor que el grupo ya tenía. Su debilidad es que mira al frente una vez cada 1.2 s; por eso su nota social es menor. Esa debilidad se atacó en el diseño con la estimación del tiempo antes de chocar (sección 4.4).

3.3. Alimentación

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Una celda (1S) con elevador a 5 V	2	4	4	2	2.80
Pack 2S de Li-Ion con BMS y reductores	3	3	4	3	3.15
Power bank USB y baterías de 9 V para los motores	4	4	2	5	**3.90**

Con una celda, entregar 15 W a 5 V pide cerca de 4.5 A de entrada, y más de 5 A con la celda descargada: los elevadores MT3608 y XL6009 no lo sostienen, y una caída de tensión corrompe la microSD.

El pack 2S alimenta todo desde una sola fuente y es recargable, pero obliga al grupo a armar y ajustar el BMS, el fusible y dos reductores antes de conectar un equipo prestado. Además, los reductores comunes no son aislados: con los dos rieles colgando del mismo pack, las tierras de la lógica y de los motores quedan unidas por el retorno, que es justo lo que el enunciado pide evitar.

El power bank entrega 5 V ya regulados y trae de fábrica la protección de su celda. Con las baterías de los motores aparte, los dos dominios no comparten ningún conductor y la separación de tierras viene dada por la propia fuente. Las baterías de 9 V se consiguen en cualquier comercio. Su debilidad es ambiental y de autonomía: son alcalinas desechables y de poca capacidad, y por eso esa alternativa tiene la nota ambiental más baja.

3.4. Salida de audio

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Jack de 3.5 mm con LM386	3	4	3	3	3.25
PWM por GPIO 18 con PAM8403	4	5	4	4	**4.25**
DAC I2S (MAX98357A)	4	3	4	4	3.75

El LM386 es clase AB, rinde cerca del 50 % y pide ocho componentes discretos para dar 0.5 W. El DAC I2S da el mejor audio, pero es más caro, más difícil de conseguir localmente y cambia la configuración de ALSA. El PAM8403 es un módulo armado, clase D ($\approx$ 85 %), que entrega 1.3 W sobre 8 Ω con un filtro de tres pasivos.

3.5. Odometría

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Solo el modelo de los motores	2	5	5	3	3.40
Encoders ópticos de rueda	4	3	3	3	3.40
MPU-6050 fusionado con el modelo	4	5	5	4	**4.40**

Estimar la posición solo con el tiempo y el ciclo de trabajo deja derivar el rumbo en cuanto las dos llantas giran distinto, y el rumbo es lo que más deforma el mapa. Los encoders miden bien la distancia, pero había que comprarlos y montarlos en motores que no traen eje para ello. El MPU-6050 ya estaba en el inventario: su giroscopio da el rumbo y su acelerómetro la dinámica de la velocidad, que además habilita el tiempo antes de chocar.

3.6. Otras decisiones

Decisión	Elegida	Alternativas y motivo
Aislamiento	Cuatro PC817	El DRV8871 acepta 3.3 V pero comparte tierra: no aísla y se descartó. El ADuM1401 aísla cuatro canales en un chip, pero es más caro y difícil de conseguir.
Acceso a GPIO	`pigpio`	`libgpiod` y WiringPi no generan PWM ni pulsos de servo con temporización estable; `pigpio` lo hace por DMA, mide el eco en microsegundos y da el bus I2C por el mismo demonio.

Decisión	Elegida	Alternativas y motivo
Servidor web	`libmicrohttpd`	nginx o lighttpd son un proceso aparte con CGI que mantener; un servidor en Python suma unos 40 MB al rootfs.
Decodificador MP3	`mpg123`	GStreamer trae decenas de MB de plugins para el mismo resultado.
Imagen base	`core-image-minimal`	`core-image-base` y `full-cmdline` arrastran utilidades y servicios que el robot no usa.
Chasis	Circular, impreso en 3D	El rectangular aprovecha mejor el espacio, pero se traba en las esquinas al girar; el circular rota sobre su eje sin barrer área extra. La impresión 3D permite integrar los apoyos de cada componente y el cierre a presión en la misma pieza.
Algoritmo de cobertura	Aleatorio con rebote	Espiral y zig-zag dependen de una odometría precisa; el rebote tolera el error acumulado y saca partido de las siete direcciones del radar.

4. DI3 — Diseño de la alternativa seleccionada

4.1. Arquitectura de hardware

Dos fuentes independientes alimentan los dos dominios: un power bank USB, la Raspberry Pi y todo lo que trabaja a 5 V; dos baterías alcalinas de 9 V en paralelo, el puente H y los motores. Las cuatro señales que cruzan del dominio lógico al de potencia —las entradas del puente H— lo hacen por optoacopladores, y las dos tierras no se tocan en ningún punto.

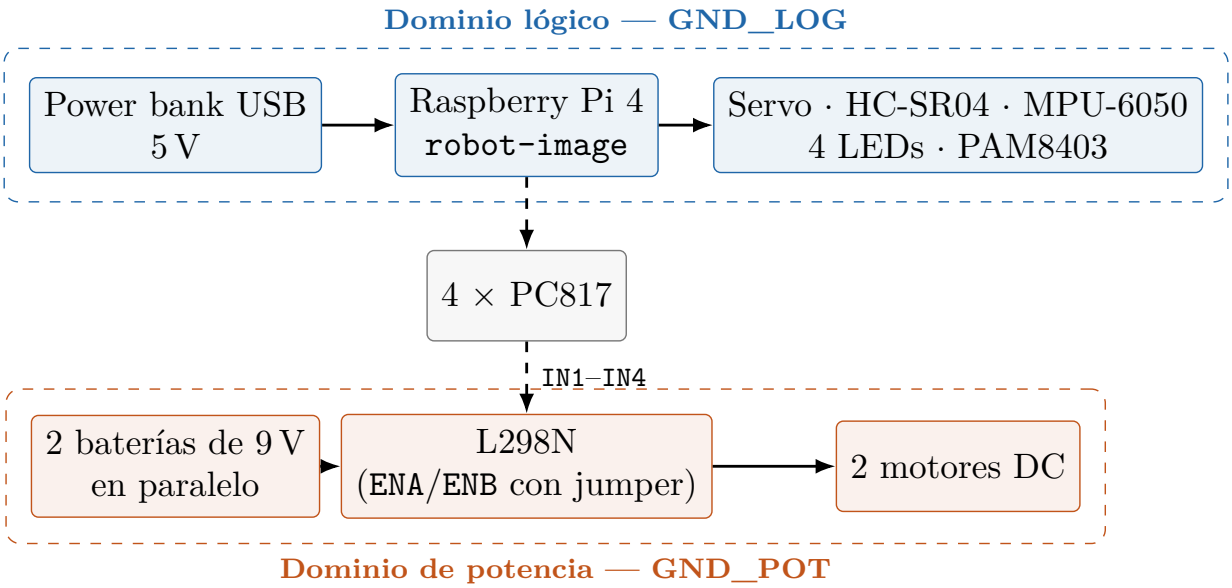


Figura 1: Arquitectura de hardware. Las líneas punteadas son señales que cruzan la barrera óptica. Los esquemas eléctricos completos están en `hardware-diagramas.pdf`.

Alimentación. Un power bank USB alimenta la Raspberry Pi por USB-C. Del pin de 5 V de la Pi salen el servo, el HC-SR04, el MPU-6050 y el amplificador: unos 1.6 A típicos. El dominio de potencia son dos baterías alcalinas de 9 V en paralelo, conectadas directo a la entrada del L298N; el regulador de 5 V del propio módulo alimenta su lógica y los pull-up de los optoacopladores. El L298N cae unos 2 V, así que a los motores les llegan cerca de 7 V. Detalle en `docs/hardware-alimentacion.md`.

Aislamiento. Un PC817 por entrada del puente H. Resistencia de entrada de 390 Ω, que fija 5 mA por canal: con cuatro canales son 20 mA, y junto con los LEDs el conector entrega 37 mA de los 50 mA que admite. El servo no cruza la barrera: pertenece al dominio lógico y su señal sale directo del GPIO 25. El L298N lleva los jumpers `ENA` y `ENB` puestos y se controla solo con `IN1`–`IN4`, lo que ahorra dos optoacopladores y 10 mA. Detalle en `docs/hardware-aislamiento.md`.

Sensado. Un HC-SR04 sobre un servo de 180° en el borde frontal, con el eje 10 cm por delante del centro. El pin `ECHO` entrega 5 V y pasa por un divisor de 1 kΩ y 2 kΩ (3.33 V) antes del GPIO. El MPU-6050 va entre las dos llantas, en un módulo GY-521 alimentado de los 5 V de la Raspberry Pi: el módulo trae su propio regulador de 3.3 V. Detalle en `docs/hardware-sensores.md`.

Indicadores y audio. Cuatro LEDs de colores distintos a 5 mA, con la resistencia calculada para cada color. El audio sale por PWM en el GPIO 18 (overlay `audremap`), pasa por un filtro RC con corte en 19 kHz y entra al PAM8403, alimentado de los 5 V de la Raspberry Pi para que el ruido de los motores no llegue al parlante.

Modelo físico. Carcasa circular de 215 mm de diámetro y 45 mm de alto, impresa en 3D en dos piezas que cierran a presión con cuatro pestañas, sin tornillos. El diámetro es el máximo que admite la cama de la impresora. Dos ruedas impresas de 30 mm entran a presión en el eje de los motores, 12 mm por delante del centro, y una bola de 16 mm sirve de apoyo atrás; el piso queda a 8 mm del suelo. Al frente, una campana lleva el aire desde una boca de 70×12 mm hasta el ventilador de succión. El modelo es paramétrico (CadQuery) y está en `modelo-3d/`; el detalle, en `docs/hardware-chasis.md`.

4.2. Arquitectura de software

Sistema operativo. La imagen `robot-image` parte de `core-image-minimal` y agrega solo lo que el robot usa; cada paquete está justificado en `docs/paquetes.md`. La capa propia `meta-robot` aporta las recetas de `librobot`, `robot-server`, `pigpio`, la configuración de ALSA y de WiFi, los ajustes de `config.txt` y la tabla de particiones. La música va en una tercera partición, de modo que ninguna supera los 200 MB.

Biblioteca. `librobot` es la única vía de acceso al hardware. Una llamada, `robot_init()`, abre la sesión e inicia dos hilos propios: la odometría a 50 Hz y el barrido del radar. El audio corre en un tercer hilo. La referencia completa está en `docs/api-librobot.md`.

Servidor. `robot-server` sirve la interfaz, expone la API y corre el hilo de navegación a 10 Hz. No espera al radar: lee la última lectura de cada ángulo cuando la necesita. Las contraseñas se guardan como resumen SHA-256 y la sesión viaja en una cookie `HttpOnly` que caduca a los 10 minutos de inactividad.

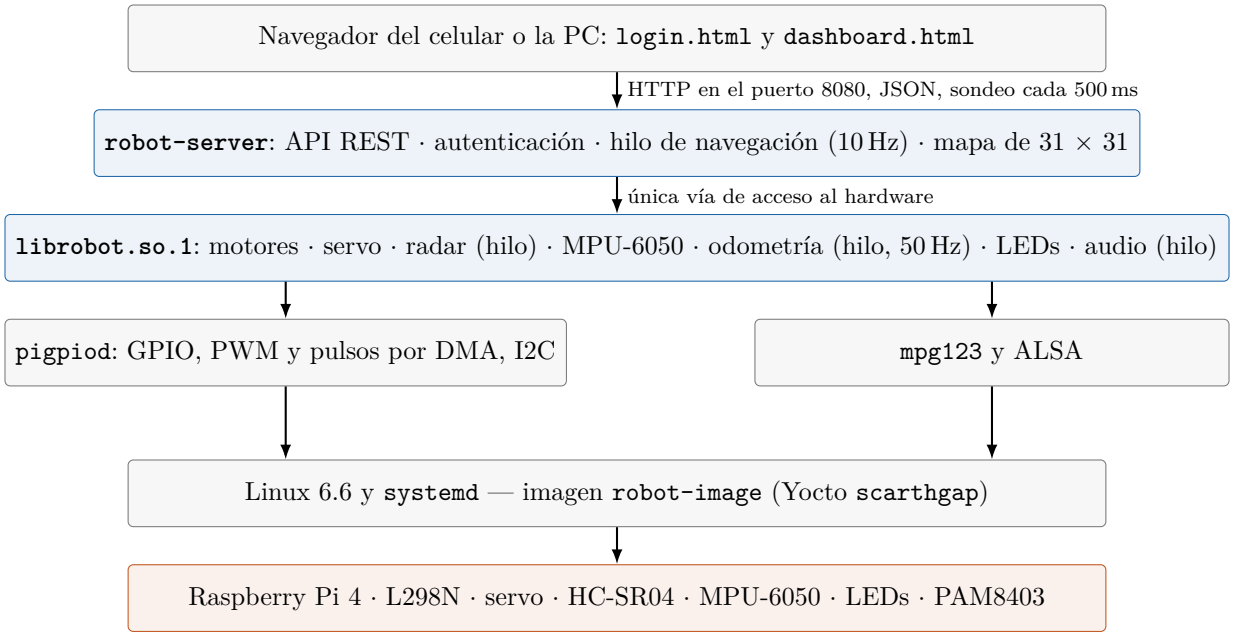


Figura 2: Arquitectura de software por capas.

Arranque. Dos unidades `systemd`, habilitadas desde las recetas. `robot-server` declara `Requires=pigpiod.service` y `Restart=on-failure` con un límite de cinco reintentos por minuto.

4.3. Navegación

El robot avanza hasta que detecta un obstáculo por cualquiera de dos vías: el tiempo antes de chocar baja de 1.2 s, o alguna lectura del cono frontal (60°, 90°, 120°) baja de 20 cm. Entonces se detiene, suena el aviso y enciende el LED rojo. En modo autónomo sigue la evasión:

1. retrocede medio segundo;
2. espera un barrido completo del radar, ya quieto;
3. elige el rumbo con más espacio entre las seis direcciones que no son el frente; las que quedan a menos de 25 cm de la mejor compiten al azar, que es la parte aleatoria del rebote;
4. gira en lazo cerrado sobre el rumbo del giroscopio hasta cubrir el ángulo elegido, no durante un tiempo fijo.

En modo manual la misma detección actúa como freno de seguridad.

4.4. Elementos propios del diseño

- **Tiempo antes de chocar.** Cada lectura frontal se congela con el avance acumulado en ese instante. Entre dos lecturas, la distancia se proyecta con lo que el robot avanzó y se divide entre la velocidad actual: $t = (d - \Delta\text{avance})/v$. La cuenta regresiva sigue bajando aunque el sensor esté mirando a un lado, y compensa la baja frecuencia de la lectura frontal del radar.
- **Fusión de la velocidad.** Un filtro complementario deja pasar la dinámica rápida del acelerómetro y toma del modelo de los motores la tendencia lenta ($\tau = 1\,\text{s}$); con los motores detenidos la velocidad se fuerza a cero. Así el sesgo del acelerómetro no se acumula.

- **La inversión se compensa en un solo punto.** Los cuatro canales del puente H van en emisor común, que invierte la señal. La inversión se compensa en un único lugar de `lib_motors.c`, y el simulador invierte igual que la placa: si alguien quita la compensación, la prueba falla.
- **El reposo es seguro.** Con los GPIO en bajo, las cuatro entradas del L298N quedan en alto por sus resistencias: con el puente habilitado eso es freno, no movimiento.
- **Desarrollo sin hardware.** Un simulador reemplaza a `pigpiod` por un robot virtual en una sala de 4×4 m, sin tocar una línea de la biblioteca. Una máquina `qemuarm64-robot` con el mismo *tune* Cortex-A72 arranca la imagen completa con los mismos binarios de espacio de usuario que corren en la Pi.
- **Imagen reproducible sin secretos.** Si falta el archivo de credenciales de WiFi, la receta usa la plantilla y avisa: un clon limpio produce la imagen sin pasos manuales, como exige R18.

4.5. Cómo responde el diseño a seguridad, costo e impacto

Aspecto	Respuesta del diseño
Seguridad	Power bank con la protección de su celda de fábrica, dos fuentes independientes, aislamiento óptico con tierras separadas, divisor en `ECHO`, freno en reposo y freno automático en modo manual, `pigpiod` con `-l` (solo interfaz local), inicio de sesión obligatorio.
Costo	Se reutilizó el sensor, el MPU-6050 y el L298N del grupo. El radar evitó comprar dos sensores; el MPU-6050 evitó los encoders; el PAM8403 reemplazó ocho componentes discretos.
Ambiental	Amplificador clase D; módulos con conectores desconectables y reutilizables; power bank recargable. Las baterías de 9 V son alcalinas desechables: es el punto débil del diseño, y al agotarse se llevan a un punto de recolección.

5. DI4 — Validación del diseño final

5.1. Estrategia

La validación se hizo en tres niveles, de menor a mayor costo de cada iteración:

1. **Simulador en el host.** `sim/prueba_librobot` ejercita cada módulo de la biblioteca contra el robot virtual y compara con la verdad del mundo simulado: 47 comprobaciones.
2. **Emulación ARM.** El mismo programa, compilado con el Toolchain-SDK, corre bajo `qemu-aarch64`; y la imagen completa arranca en la máquina `qemuarm64-robot`.
3. **Target.** La imagen en la Raspberry Pi 4 con el robot armado, siguiendo los procedimientos de verificación de `docs/`.

5.2. Matriz de cumplimiento

Id.	Cómo se validó	Resultado	Evidencia
R1	Dos minutos en modo autónomo sobre el simulador: 18 evasiones y ningún choque. Prueba de campo en un área delimitada.	Cumple	`docs/navegacion-radar.md`
R2	Barrido completo contra el mundo simulado: los siete ángulos a menos de 2 cm de la distancia real, a 5 lecturas por segundo. Un HC-SR04 sobre el servo y un MPU-6050.	Cumple, con desviación acordada	`sim/prueba_librobot.c`
R3	Avance, retroceso, giros y detención verificados en el simulador; sentido de giro confirmado con el robot elevado. Velocidad fija, sin PWM.	Cumple, con desviación acordada	`docs/evidencias/ejecucion-cruzada-qemu.md`
R4	Cada LED cambia de estado al ordenarlo y sigue al modo activo.	Cumple	`docs/evidencias/servidor-web.md`
R5	Reproducción en un hilo propio mientras el hilo de navegación corre.	Cumple	`lib/lib_audio.c`
R6	Listar, reproducir, pausar, detener y volumen por HTTP.	Cumple	`sim/prueba_api.sh`
R7	Tono de prueba y sonido de evento por el parlante del robot.	Cumple	`docs/hardware-sensores.md`
R8	Los cuatro eventos disparan su sonido; el aviso pausa la música y la reanuda.	Cumple	`docs/api-librobot.md`
R9	Cambio de modo y movimiento manual por HTTP; los controles manuales se bloquean en autónomo.	Cumple	`docs/evidencias/servidor-web.md`
R10	`/api/status` entrega modo, sensores, LEDs, audio y mapa en una sola consulta, cada 500 ms.	Cumple	`docs/evidencias/servidor-web.md`
R11	La grilla crece en tiempo real: de 79 a 85 celdas visitadas en 6 s.	Cumple	`docs/evidencias/servidor-web.md`
R12	Credenciales inválidas y consultas sin sesión devuelven 401.	Cumple	`docs/evidencias/servidor-web.md`
R13	Tras un `kill -9`, el servicio vuelve con otro PID y `NRestarts=1`.	Cumple	`docs/evidencias/systemd-reinicio.md`
R14	El servidor declara `NEEDED librobot.so.1` y no toca GPIO.	Cumple	`docs/evidencias/binarios-target.md`
R15	Los logs muestran `aarch64-poky-linux-gcc`; los binarios son ELF ARM aarch64 y no ejecutan en el host.	Cumple	`docs/evidencias/`
R16	Imagen sobre `core-image-minimal`; cada paquete agregado, listado y justificado.	Cumple	`docs/paquetes.md`

Id.	Cómo se validó	Resultado	Evidencia
R17	Ambas recetas heredan `cmake`; la biblioteca se empaqueta en `librobot` y `librobot-dev`.	Cumple	`meta-robot/`
R18	`bitbake robot-image` produce la imagen desde un clon limpio.	Cumple	`docs/yocto-setup.md`
R19	Ver la sección 5.4.	Ver tabla	`docs/metricas.md`
RO1	Tres minutos en autónomo sobre el simulador, con una grada que el HC-SR04 no ve: una detección con evasión y ninguna caída. Falta montar los sensores en el robot.	Cumple en simulador	`docs/opcionales.md`
RO2	Con una meta de 12 s el robot se detuvo a los 12.5 s, sonó el aviso y el panel marcó el ciclo completo.	Cumple	`docs/opcionales.md`
RO3	La playlist se guardó, sobrevivió a un reinicio del servidor y se reprodujo en orden y en bucle.	Cumple	`docs/opcionales.md`

Desviaciones acordadas con el profesor. En dos requerimientos el diseño se aparta de la letra del enunciado. Ambos casos se consultaron con el profesor.

- **R3, velocidad sin PWM.** El robot avanza, retrocede, gira hacia los dos lados y se detiene, que es el control diferencial que se pide. Lo que no usa es PWM: los motores van a velocidad fija. Son motorreductores lentos y de alto torque (30 rpm a 6 V y 60 rpm a 12 V; 110 y 200 oz-in a rotor bloqueado) que trabajan a unos 9 V, lo que da la batería. Con ruedas de 30 mm el robot avanza a unos 5 cm/s; con la tensión recortada por la PWM el robot no se mueve (hallazgo 3 de la sección 5.6). Se acordó con el profesor que la PWM no era necesaria.
- **R2, sensores.** El robot lleva un solo sensor de proximidad, un HC-SR04 que sobre el servo cubre las detecciones frontal y lateral, y un MPU-6050 que aporta el giroscopio. El profesor lo consideró suficiente: lo que busca es que el robot tenga dos sensores.

5.3. Validación de la seguridad

Antes de conectar la Raspberry Pi se ejecutaron, en orden, los procedimientos de verificación de `docs/hardware-aislamiento.md` y `docs/hardware-alimentacion.md`. Todas las verificaciones pasaron:

Verificación	Criterio	Resultado
Continuidad entre `GND_LOG` y `GND_POT`, todo desenergizado	Circuito abierto ($> 1\,\mathrm{M}\Omega$)	Pasó
Continuidad entre el pin de 5 V de la Raspberry Pi y `VS` del L298N	Circuito abierto	Pasó
Salida de 5 V del L298N, que alimenta `VSS` y los pull-up de los optoacopladores	$5 \pm 0.25\,\mathrm{V}$	Pasó
Entradas del L298N con los optoacopladores en reposo	$\approx 5\,\mathrm{V}$: motores frenados	Pasó
Salida del divisor de `ECHO` al disparar el sensor	3.3 V como máximo	Pasó
Raspberry Pi navegando con audio, alimentada del power bank	Sin reinicios ni subtensión	Pasó

La separación entre los dos dominios no depende de un regulador: cada uno tiene su propia fuente, y las únicas señales que cruzan lo hacen por los optoacopladores.

5.4. Validación de la eficiencia

Métrica	Referencia	Medido	Método
Rootfs	$\leq 200\,\mathrm{MB}$	129 MB	`du` sobre el rootfs de la imagen construida. La partición raíz es de tamaño fijo, 180 MiB.
Tiempo de arranque	$\leq 15\,\mathrm{s}$	19.2 s	Instante en que `robot-server` escucha en el puerto 8080, en la Raspberry Pi 4 con WiFi.
RAM en operación	—	97 MB de 1845 MB	`/proc/meminfo`; `robot-server` ocupa 5.2 MB (`VmRSS`).
CPU en operación	—	3.6 % de 4 núcleos	`/proc/stat` durante 60 s; `robot-server` usa el 4.5 % de un núcleo.

Medido sobre la imagen final con el robot armado, en modo autónomo, con música y el panel consultando el estado cada 500 ms. El rootfs queda en el 65 % del presupuesto, y la RAM y la CPU dejan margen de sobra. El arranque excede la referencia por 4.2 s: `robot-server` espera a que la red esté operativa (`network-online.target`), y la búsqueda del punto de acceso WiFi se lleva casi 7 s; desde que hay red, el servidor escucha en 1.5 s. Se considera una desviación justificada, porque sin red el panel de control no es alcanzable y el servicio no estaría operativo en el sentido del enunciado. El método completo está en `docs/metricas.md`.

5.5. Validación del costo y del impacto

- **Costo.** El grupo gastó 134 USD, por encima del límite superior de la estimación (99 USD). La diferencia tiene dos causas. La primera es que 44 USD no terminaron en el robot: 24 de los motores y el ventilador del primer diseño, que se descartaron al rehacerlo, y 20 de la aspiradora que se desarmó para hacerle ingeniería inversa. La segunda es que la impresión 3D costó 50 USD, el doble del máximo previsto para el bloque de chasis y acabado. Lo que quedó en el robot suma 90 USD, dentro del intervalo estimado. La lección es cerrar el diseño mecánico antes de comprar los motores y cotizar la impresión antes de fijar el chasis.

- **Impacto ambiental.** Todos los módulos quedaron con conectores desconectables; la Raspberry Pi se devuelve sin soldaduras ni adhesivos; el power bank es recargable. Las dos baterías de 9 V son alcalinas desechables, el mayor impacto del diseño final: se entregan a un punto de recolección al agotarse.

5.6. Hallazgos de la validación y acciones correctivas

Validar no fue solo confirmar: varias pruebas cambiaron el diseño.

	Hallazgo	**Causa**	**Acción**
1	El límite de reintentos de `systemd` no existía.	`StartLimitIntervalSec` estaba en `[Service]`; desde systemd 229 va en `[Unit]` y en otro lugar se ignora en silencio.	Se movió a `[Unit]`. Lo detectó `systemd-analyze verify`.
2	Con los optoacopladores, los motores girarían al revés.	El PC817 en emisor común invierte la señal.	Compensación en un único punto de `lib_motors.c`; el simulador invierte igual que la placa, así que quitarla hace fallar la prueba.
3	Con PWM de 1 kHz los motores no se movieron.	El PC817 con pull-up de 4.7 kΩ tarda decenas de microsegundos en apagarse; sumado a los 2 V que cae el L298N, a ciclos medios el motor no alcanza su tensión de arranque.	Velocidad fija, sin PWM. Los motores dan 60 rpm a 12 V y trabajan a unos 9 V: con la tensión recortada no mueven el robot. Se consultó con el profesor y se acordó la desviación.
4	El servidor arrancaba a los 130 s.	`eth0` sin cable y dos clientes DHCP sobre `wlan0` agotaban la espera de red.	Ethernet marcada como no requerida y un solo cliente DHCP: 17 s.
5	El vaivén del servo sacudía el sensor y el chasis.	El servo iba a su velocidad máxima entre paradas.	Rampa a un tercio de la velocidad, con un pulso intermedio cada 20 ms.
6	El *build* colgaba el host al 67 %.	Tres compilaciones pesadas a la vez agotaron 7 GB de RAM y el swap.	Dos hilos y regulación por presión de recursos (`BB_PRESSURE_MAX_MEMORY`).
7	Una Raspberry Pi dejó de funcionar durante el desarrollo.	No se determinó; se presume la antigüedad de la tarjeta.	Se reportó al profesor, quien la reemplazó, y el proyecto continuó con la nueva.

5.7. Limitaciones conocidas

- Los motores van a velocidad fija, sin PWM: el robot gira sobre su eje, no en curvas de radio variable. Es una desviación acordada con el profesor.
- El radar usa un solo sensor de proximidad para cubrir las detecciones frontal y lateral; el otro sensor del robot es el MPU-6050. También es una desviación acordada. Los GPIO 22 y 10 quedaron libres para un segundo HC-SR04 fijo al frente.

- Sin encoders, la distancia recorrida depende de la calibración del modelo de los motores; el mapa sirve para la sesión, no como posición absoluta.
- Una lectura sin eco no significa camino libre: una pared oblicua desvía el pulso. El mapa no usa esas lecturas para despejar celdas.
- Las baterías de 9 V tienen poca capacidad y su tensión cae bajo la corriente de los motores: la autonomía de la tracción es corta y conviene tener repuestos a mano.
- Los sensores de desnivel se probaron solo en el simulador: falta montarlos y ajustar su umbral sobre el piso de la demostración.
- La meta del ciclo de limpieza vive en memoria: vuelve a "sin límite" al reiniciar el robot.

6. Conclusiones

El diseño cumple los requerimientos obligatorios, con dos desviaciones acordadas con el profesor (velocidad fija en vez de PWM, y un sensor de proximidad sobre un servo junto con un giroscopio), con un rootfs de 129 MB y un modelo físico construido con el inventario del grupo y una compra acotada. Las decisiones que más pesaron —radar sobre servo, dos fuentes independientes, MPU-6050 fusionado con el modelo de los motores— se tomaron comparando alternativas con criterios técnicos, económicos, ambientales y sociales, y cada una dejó una debilidad conocida que el propio diseño atiende.

La validación en tres niveles permitió avanzar el software antes de tener el hardware, pero también mostró su límite: el problema de la PWM a través del optoacoplador solo apareció en la placa real. El diseño de un sistema embebido se cierra en el hardware.

7. Referencias

- J. González Gómez, *Especificación Proyecto I*, CE-1113 Sistemas Empotrados, Tecnológico de Costa Rica, 2026.
- Yocto Project, *Reference Manual* y *Development Tasks Manual*, versión 5.0 (`scarthgap`).
- Raspberry Pi Ltd., documentación de la Raspberry Pi 4 Model B: GPIO, `config.txt` y *device tree overlays*.
- `pigpio`, documentación de la interfaz `pigpiod_if2`.
- Hojas de datos: Sharp PC817, STMicroelectronics L298N, InvenSense MPU-6050, Diodes Inc. PAM8403, HC-SR04.
- Repositorio del proyecto: `https://github.com/Randall-BL/EmpotradosProyecto1`.